J. Phys. Soc. Jpn.
Downloaded from journals.jps.jp by Cornell University on 08/19/20

Journal of the Physical Society of Japan **88**, 061007 (2019)
https://doi.org/10.7566/JPSJ.88.061007

Quantum Annealing amid Local Ruggedness and Global Frustration

James King*, Sheir Yarkoni, Jack Raymond, Isil Ozfidan, Andrew D. King, Mayssam Mohammadi Nevisi, Jeremy P. Hilton, and Catherine C. McGeoch

D-Wave Systems, Inc., Canada

(Received September 1, 2018; accepted October 26, 2018; published online March 1, 2019)

Quantum annealers are designed to utilize quantum tunneling to find good solutions to hard optimization problems. When constructing a family of synthetic inputs to test the potential of a quantum annealing platform, one should therefore ensure that the inputs a) have locally rugged energy landscapes so that solvers can benefit from quantum tunneling, and b) are hard optimization problems with global frustration so that they are computationally meaningful.

A recent Google study [Phys. Rev. X **6**, 031015 (2016)] introduced such an input class, but the amount of global frustration is limited and they utilize gadgets that are tailored to deceive classical annealing algorithms. In this study we extend their work by introducing a problem class that contains more global frustration and uses only simple clusters to induce ruggedness. Further, our problem class has tunable levels of ruggedness and frustration, making it ideal for the analysis of quantum annealers and classical approximations of quantum annealing.

Like the Google inputs, these inputs are built from easy-to-solve logical problems that can be extracted by tailored solvers. However, they provide a meaningful testbed for solvers agnostic to the planted problem class. We compare the latest-generation D-Wave quantum processing unit (QPU) to classical alternatives, examining both optimization and sampling, and measure the response of these solvers to increasing levels of local ruggedness.

1. Introduction

Quantum annealers are designed to take advantage of quantum tunneling to find good solutions to hard optimization problems. When constructing a family of synthetic inputs to test the potential of a quantum annealing platform, one should therefore ensure that the inputs a) are such that solvers can benefit from quantum tunneling, and b) are hard optimization problems with global frustration.

For a solver to benefit from quantum tunneling, the energy landscape associated with the input must have tall, thin energy barriers. For an input to be computationally hard, the input must have constraints that interact with each other in nontrivial ways.

Quantum processing units (QPUs) developed by D-Wave Systems that use the quantum annealing algorithm have been commercially available since 2011. These QPUs solve Ising model inputs defined on the underlying working graph of the chip. There have been various efforts to evaluate the performance of the D-Wave systems using synthetic inputs generated randomly from different distributions, or *input classes*.

This study has two main contributions: to propose a new problem class ideal for evaluating quantum annealing processors, and to use this problem class to evaluate the latest-generation D-Wave QPU, the D-Wave 2000Q.

1.1 Proposing a new problem class

Previous evaluations of D-Wave QPUs have used problem classes that benefit either too little or too much from quantum tunneling to be ideal for evaluating quantum annealers.

On one side of this spectrum we have problems such as random unstructured ± 1 problems on the Chimera topology native to D-Wave QPUs. These were used by Rønnow et al.[1] in their evaluation of the D-Wave Two QPU in 2014, but they are now known[2] to lack a finite-temperature phase transition, meaning that quantum tunneling is unlikely to play a significant role when solving them.

On the other side of the spectrum, Denchev et al.[3] recently introduced an input class designed to benefit immensely from quantum tunneling. We refer to these inputs as Google problems. Their study showed a massive speed increase (up to 100 million times faster) of a D-Wave 2X system over simulated annealing (SA) and quantum Monte Carlo (QMC), also known as *simulated quantum annealing*. This provided strong evidence for the ability of quantum annealing to leverage quantum tunneling in a computationally relevant way. However, the spin-glass backbones of the Google problems are easy to solve, meaning that a) the problems have limited relevance to real-world problems, and b) certain algorithms omitted from that study can solve them with relative ease.[4]

In this study we provide a problem class that aims to retain the advantages of Google problems while being more reflective of real-world problems. They are more reflective of real-world problems because, rather than relying too heavily on finely-tuned gadgets, they derive much of their computational hardness from larger spin-glass backbones with planted frustration.

1.2 Evaluation of the 2000-qubit D-Wave QPU

We use this new problem class to evaluate the latest-generation D-Wave QPU. We measure its performance in absolute terms and we analyze its response to the ruggedness parameters of the problem class.

The software competition we consider is much stronger than that considered by Denchev et al.,[3] and includes GPU implementations of SA, QMC, and SVMC (spin vector Monte Carlo, a mean-field approximation to QMC), and also includes Selby's implementation[5,6] of the Hamze–de Freitas–Selby (HFS) algorithm.[5,7] In the study of Mandrà et al.[4] that used a wide array of algorithms to solve Google problems, Selby's implementation of HFS was the fastest software solver in terms of both scaling and absolute speed.

We find that the D-Wave QPU is able to find ground states significantly faster than the software competition. We also consider the problem of sampling from ground states

©2019 The Physical Society of Japan

J. Phys. Soc. Jpn.
Downloaded from journals.jps.jp by Cornell University on 08/19/20

and find that the D-Wave QPU maintains a similar advantage and does not struggle to find a diverse set of optimal solutions.

1.3 Limitations of the study

Our problems are synthetic and are generated by first creating a logical problem on a square lattice, then embedding this logical problem on the Chimera topology native to D-Wave QPUs by expanding logical variables to clusters of variables. This simple embedding procedure can easily be detected and reversed, and therefore our inputs can be solved easily by tailored solvers. Because the logical problems are planar Ising models without fields, they can actually be solved in polynomial time.[8)]

We exclude logical solvers tailored to this problem class from our set of competition solvers because they deliberately avoid the issue of interest, i.e., optimization of problems with locally rugged energy landscapes having global frustration. Attacking the logical problems directly removes the local ruggedness from the energy landscape and leaves the much smaller spin-glass backbone with frustration but little ruggedness; further, because the clusters are trivial to find because of the restrictions of the Chimera topology, it does this at negligible cost to the solver.

The point of our study and the Google study[3)] is to consider the future scenario in which clusters are hard to identify because QPUs have much denser working graphs. Identifying clusters of vertices to flip together is generally a computationally hard problem closely related to graph clustering[9)] and community detection;[10)] for our inputs and the Google inputs, it is only easy because, in the Chimera topology, the densest clusters are exactly the unit tiles and are therefore trivial to find.

While we exclude solvers that are specifically designed to extract our logical problems because they are not reflective of any algorithm that will be viable in the future, we note that general cluster-detecting algorithms are fair game. Specifically, Swendsen and Wang's replica Monte Carlo with cluster flips[11)] and variations thereof[12,13)] are well-suited to the problem class without being specifically tailored to it. Similarly, Selby's implementation[6)] of the HFS algorithm,[5,7)] while tailored to the Chimera topology, is not tailored to the problem class and is therefore included in our study.

We wish to emphasize that this study makes no attempt to measure scaling advantage of one solver over another or to measure quantum speedup of any kind.

1.4 Subsequent work

An earlier version of this manuscript was presented on arXiv in early 2017.[14)] Since then there have been two notable extensions to our work. In one, Mandrà and Katzgraber[15)] explored the weakening of cluster bonds which can make our inputs harder for tailored solvers. With their inputs, they found an absolute speed advantage of D-Wave QPUs over tailored classical solvers. In another, Albash and Lidar[16)] added gadgets to our inputs that made quantum annealing benefit from longer annealing times, enabling the first experimental demonstration of a scaling advantage of quantum annealing over simulated (thermal) annealing.

1.5 Outline

The remainder of the paper is organized as follows. In Sect. 2 we provide a description of the 2000-qubit D-Wave system and a history of D-Wave QPUs. In Sect. 3 we present the problem class analyzed in this paper and discuss the concept of ruggedness and its relevance to optimization problems. In Sect. 4 we discuss the software solvers used in our evaluations, as well as notable solvers that were not suitable. In Sect. 5 we present our experimental results on optimization. In Sect. 6 we present our experimental results on sampling from ground states. In Sect. 7 we provide further discussion and conclude the paper.

2. D-Wave Quantum Processing Units

We start with an overview of D-Wave design features and introduce notation that will be used throughout. For details about underlying technologies see Bunyk et al.,[17)] Dickson et al.,[18)] Harris et al.,[19)] Johnson et al.,[20)] or Lanting et al.[21)]

2.1 Ising minimization

D-Wave annealing-based quantum processors are designed to find minimum-cost solutions to the Ising minimization (IM) problem, defined on a graph $G = (V, E)$ as follows. Given a collection of fields $h = \{h_i : i \in V\}$ and couplings $J = \{J_{ij} : (i,j) \in E\}$, assign values from $\{-1, +1\}$ to *n spin variables* $s = \{s_i\}$ so as to minimize the *energy function*

$$E(s) = \sum_{i \in V} h_i s_i + \sum_{(i,j) \in E} J_{ij} s_i s_j. \tag{1}$$

The spin variables s can be interpreted as magnetic poles in a physical particle system; in this context, negative J_{ij} is *ferromagnetic* and positive J_{ij} is *antiferromagnetic*, the optimal solution is called a *ground state*, and nonoptimal solutions are *excited states*. IM instances can be trivially transformed to Quadratic Unconstrained Boolean Optimization (QUBO) instances defined on integers $s = \{0, 1\}$, or to Maximum Weighted 2-Satisfiability (MAX W2SAT) instances defined on Booleans $s = \{\text{true}, \text{false}\}$, all of which are NP-hard.

2.2 Chimera topology

The native connectivity topology for the D-Wave QPU is based on a C_{16} *Chimera graph* containing 2048 vertices (qubits) and 6016 edges (couplers).

A Chimera graph of size C_s is an $s \times s$ grid of Chimera cells (also called unit tiles or unit cells), each containing a complete bipartite graph on 8 vertices (a $K_{4,4}$). Each vertex is connected to its four neighbors inside the cell as well as two neighbors (north/south or east/west) outside the cell: therefore every vertex has degree 6 excluding boundary vertices.

In this study, as in others, we vary the problem size using square subgraphs of the full graph, from size C_4 (128 vertices) up to C_{16} (2048 vertices). Note that the number of problem variables $n = 8s^2$ grows quadratically with Chimera size. The reason we measure algorithm performance as a function of the Chimera size and not the number of qubits is that problem difficulty tends to scale exponentially with the Chimera size, i.e., with the square root of the number of qubits, since the treewidth of a Chimera graph C_s is linear in s.[22,23)]

©2019 The Physical Society of Japan

J. Phys. Soc. Jpn.
Downloaded from journals.jps.jp by Cornell University on 08/19/20

Because the chip fabrication and trapped magnetic flux leave some small number of qubits unusable, each processor has a specific *hardware working graph* $H \subset C_{16}$. The qubit yield — the fraction of qubits that are operational — is typically around 98% for the 2000-qubit D-Wave system whereas 95% was typical for the D-Wave 2X. The working graph used in this study has 2035 working qubits out of 2048.

2.3 *Quantum annealing*

D-Wave processors solve Ising problems by *quantum annealing* (QA) in the form proposed by Kadowaki and Nishimori.[24] The QA algorithm is implemented in hardware using a framework of analog control devices to manipulate a collection of qubit states according to a time-dependent Hamiltonian shown below.

$$\mathcal{H}(t) = A(t) \cdot \mathcal{H}_{\text{init}} + B(t) \cdot \mathcal{H}_{\text{prob}}. \tag{2}$$

QA carries out a gradual transition in time $t : 0 \rightarrow t_a$, from an initial ground state in $\mathcal{H}_{\text{init}}$, to a state described by the *problem Hamiltonian* $\mathcal{H}_{\text{prob}} = \sum_i h_i \sigma_i^z + \sum_{ij} J_{ij} \sigma_i^z \sigma_j^z$. The problem Hamiltonian matches the energy function (1), so that a ground state for $\mathcal{H}_{\text{prob}}$ is a minimum-cost solution to $E(s)$.

QA is closely related to *adiabatic quantum computing* (AQC). The AQC model of computation was proposed by Farhi et al.[25] who showed that if the transition is carried out slowly enough the algorithm will find a ground state (i.e., an optimal solution) with high probability.

Theoretical guarantees about solution times for quantum algorithms (found in Ref. 25) assume that the computation takes place in an ideal closed system, perfectly isolated from energy interference from ambient surroundings. The 2000-qubit D-Wave chip is housed in a highly shielded chamber and cooled to near absolute zero; nevertheless, as is the case with any real-world quantum device, it must suffer some amount of interference, which has the general effect of reducing the probability of landing in a ground state. Thus, theoretical guarantees on performance may not apply to these systems. We consider any D-Wave QPU to be a *heuristic* solver, which requires empirical approaches to performance analysis.

The D-Wave QPU studied here contains 2035 active qubits (quantum bits) and 5912 active couplers made of microscopic loops of niobium connected to a large and complex analog control system via an arrangement of Josephson Junctions. Thermometry on the refrigerator of the D-Wave QPU and fits of single qubit measurements to a thermodynamic model indicate that $T \lesssim 15$ mK. When cooled to temperatures below 9.3 K, niobium becomes a superconductor and is capable of displaying quantum properties including superposition, entanglement, and quantum tunneling. Because of these properties, the qubits on the chip behave as a quantum mechanical particle process that carries out a transition from initial state described by $\mathcal{H}_{\text{init}}$ to a problem state described by $\mathcal{H}_{\text{prob}}$.[18,21,26]

2.4 *Modeling performance*

Given input instance (h, J), a D-Wave computation involves the following steps.

(1) **Program.** Load (h, J) onto the chip; denote the elapsed programming/initialization time t_i.
(2) **Anneal.** Carry out the QA algorithm. Anneal time t_a can be set by user to some value $5\,\mu\text{s} \leq t_a \leq 1000\,\mu\text{s}$.
(3) **Read.** Record qubit states to obtain a solution; denote the elapsed readout time t_r.
(4) **Repeat.** Repeat steps b and c k times to obtain a sample of k solutions.

We define *sample time* t_s and *total time* T as follows:

$$t_s = (t_a + t_r) \tag{3}$$
$$T = t_i + k t_s.$$

For the D-Wave system studied in this paper, the median programming time t_i is 9.5 ms and the median readout time t_r is 123 µs.

In this study, both for software solvers and for the D-Wave QPU, we typically report annealing time rather than total time. Annealing time is the measure of the algorithm proper, and measuring total time often obscures trends in data. Scaling plots are particularly susceptible to this because the overhead of programming time makes scaling — typically presented on a semilog plot — look totally flat except for an uptick at the very largest problem sizes. Further, we are most interested in the future potential of D-Wave QPUs, and we expect that programming time and readout time will be reduced to small fractions of their current values; minimum annealing times will similarly be reduced, allowing us better control over the algorithm parameters. For reference, since many people will be interested in total wall clock time, rather than annealing time, a 1000 times speedup over software solvers in annealing time, typical for the D-Wave QPU in this study, translates roughly to a 30 times speedup in total wall clock time including programming and readout.

System characteristics of D-Wave QPUs such as yield can vary within a generation. If we compare this specific 2000-qubit D-Wave system to the specific D-Wave 2X QPU studied in 2015,[27] programming time has decreased by 20%, readout is three times faster, and yield has improved from 95 to 99%.

3. Frustrated Cluster Loop Problems

3.1 *Ruggedness and clusters*

Ruggedness is a feature of certain optimization problems — more specifically their energy landscapes — characterized by tall energy barriers and many local optima.[28,29] Typically, rugged problems are harder to solve, particularly with Markov chain Monte Carlo (MCMC) methods.[30,31] In the late 1980s, when ruggedness was first being explored in the context of evolutionary biology and bio-inspired computing, Kauffman's NK model was put forward as a model with tunable ruggedness inspired by genetic fitness functions under varying degrees of *epistasis*, or how many other genetic loci affect the fitness contribution of a given locus.[32–34] The tunable ruggedness of the NK model has proved very valuable in the study of optimization heuristics, particularly evolutionary algorithms.[35]

Closely related to ruggedness is the analysis of spin overlap, in which landscape features are inferred from the distribution of overlap of two random states sampled from the Boltzmann distribution.[36,37] Tall, thin peaks in the spin overlap distribution tend to correspond to tall, thin energy barriers; the presence of these features correlates not only with ruggedness and classical hardness, but also with

©2019 The Physical Society of Japan

J. Phys. Soc. Jpn.
Downloaded from journals.jps.jp by Cornell University on 08/19/20

applicability of quantum annealing, since quantum tunneling is likely to be a useful computational resource in the presence of these tall, thin barriers. Zhu et al.[38] have used spin overlap features to predict whether a problem can be solved by QA more efficiently than by SA, showing promising preliminary results for optimization problems such as weighted partial MAX-2SAT, minimum vertex cover, satisfiability, graph partitioning, circuit fault diagnosis, and certain spin-glass instances.[38,39] This work points to the potential of quantum annealers to have a place in portfolio solvers[40] and hybrid algorithms running on heterogeneous computing systems alongside CPUs, GPUs, and other coprocessors.[41,42]

To induce ruggedness using tall, thin energy barriers, Denchev et al.[3] used ferromagnetically coupled unit tiles as clusters. Flipping such a cluster in the absence of fields or external couplings involves jumping over or tunneling through an energy barrier that is 16 Ising units high and has a width of 8 in Hamming space. Denchev et al.[3] actually go beyond using single-tile clusters and use two-tile gadgets studied previously by Boixo et al.[26] This gadget is made up of two clusters that form a deceptive trap to draw annealers into a local minimum using local fields; annealers must then go over or through an energy barrier to reach the gadget's ground state.

Instead of using two-cluster gadgets, we simply use single-cell ferromagnetic clusters to induce ruggedness, leaving us with a simpler problem class.

3.2 FCL problem generation

We create local ruggedness by treating unit cells of the Chimera graph as ferromagnetically-coupled clusters. We create global frustration by joining these clusters together using a problem generated on the logical graph of clusters. This creates an energy landscape that is macroscopically interesting and in which the clusters induce wells separated by tall, thin energy barriers.

The logical graph of clusters is a square lattice, with a logical 16×16 lattice of clusters spanning the working graph of a 2000-qubit D-Wave QPU. Note that a 16×16 logical lattice is significantly larger than the largest logical lattice, 4×4, of the Google problems considered by Denchev et al.[3] — their two-tile gadgets take up more space than our one-tile clusters and the D-Wave 2X has a smaller working graph than the 2000-qubit D-Wave system. These larger logical graphs in the problems we consider mean that the spin-glass backbones of these problems are significantly more computationally challenging.

The problems we generate on the logical graph are *frustrated loop problems*. These are constraint satisfaction problems first used in the evaluation of D-Wave QPUs by Hen et al.[43] and modified to allow precision limits by King et al.[44] In a frustrated loop problem, a number of random cycles are generated, and for each cycle, a single coupler is frustrated subject to a planted solution. The final problem Hamiltonian is simply the sum of cycle Hamiltonians, making the input class similar to other satisfiability problems, but with the advantage of being generated on a specified topology, making it an ideal problem class to study fixed-topology processors such as D-Wave QPUs.

We refer to the final inputs, after combining clusters with logical frustrated loop problems, as *frustrated cluster loop* (FCL) problems. For a given Chimera graph G_C that may or may not have missing qubits or couplers, an FCL problem is generated from three parameters, α (the clauses-to-variables ratio), ρ (the range, or precision), and $R \geq \rho$ (the ruggedness) as follows:

(1) Define each unit cell as a *logical spin* if it has no missing qubits or couplers. Use $c(v)$ to denote the logical spin index corresponding to qubit v.
(2) Wherever all four couplers connecting two logical spins are present, define these couplers as a *logical coupler*.
(3) Define the logical graph G_L as the graph comprising the logical spins and logical couplers.
(4) Generate a range-bounded frustrated loop problem Hamiltonian (h_L, J_L) on G_L using parameters α and ρ as per King et al.[44] (note that h_L is the zero vector).
(5) Define the native Chimera Hamiltonian (h_C, J_C) with h_C as the zero vector and J_C as:

$$J_C(u,v) = \begin{cases} -1, & \text{if } c(u) = c(v) \\ \dfrac{1}{R} \cdot J_L(c(u), c(v)), & \text{otherwise} \end{cases}.$$

It is worth repeating that these Hamiltonians have no fields (i.e., h_L and h_C are both zero vectors). Note also that in-tile couplings in J_C are all -1 and inter-tile couplings take values in

$$\{j/R \mid j \in \{-R, -R+1, \ldots, R\}\}.$$

Since we ensure that $\rho \leq R$, and $\rho \geq |j|$ for any logical coupling j, all couplings in J_C are in the range $[-1, 1]$.

The logical frustrated loop problems may be disconnected and have multiple components; we reject such disconnected inputs at generation time.

While these problems are large enough to span the entire working graph of the latest D-Wave QPUs, the repetition code inherent in logical couplers and spins makes them relatively robust to analog errors.[45]

3.3 Problem class parameters

The FCL problem class has three parameters: the clauses-to-variables ratio α, the range ρ, and the ruggedness R. We would like to restrict our experiments to the most interesting region of the parameter space.

First we aim to determine the value of α that maximizes the difficulty of the logical problem. If α is too low, a problem is underconstrained and is easy to solve. If α is too high, the planted solution is expressed too strongly and the problem's features approach those of a ferromagnet, making it easy to solve. The difficulty of the logical problem depends only on α and ρ, not on R. For various values of ρ, we perform a sweep of α to determine the value that maximizes the hardness of the logical problem (see Fig. 1).

The impacts of ρ are more nuanced. First, the value of ρ provides an upper bound on the limit of α because packing in more loops eventually raises the maximum coupler range. Second, for a fixed value of α, problems with a lower ρ value have their loops spread out more evenly over the logical spins. Finally, for the native problem, coupler values are scaled down by a factor of $R \geq \rho$ so that inter-tile couplings are in the range $[-1, 1]$ (in-tile couplings are always -1). Thus higher values of ρ constrain R to be higher, and make problems more locally rugged relative to the global

©2019 The Physical Society of Japan

J. Phys. Soc. Jpn.
Downloaded from journals.jps.jp by Cornell University on 08/19/20

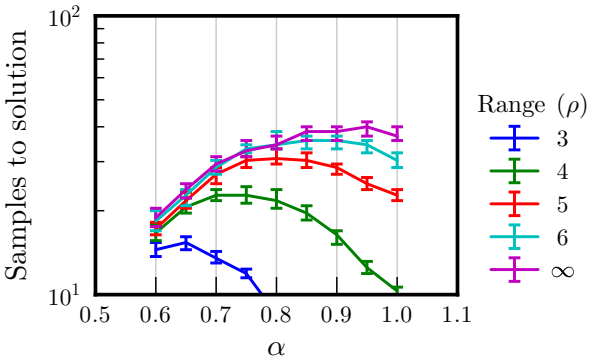


Fig. 1. (Color online) Logical problem difficulty as measured by expected samples to solution for simulated annealing. Error bars show the 95% confidence intervals for the medians, grouped over α and ρ. Difficulty is maximized at $\alpha = 0.65$ for precision 3, $\alpha = 0.75$ for precision 4, $\alpha = 0.8$ for precision 5, and $\alpha = 0.85$ for precision 6.

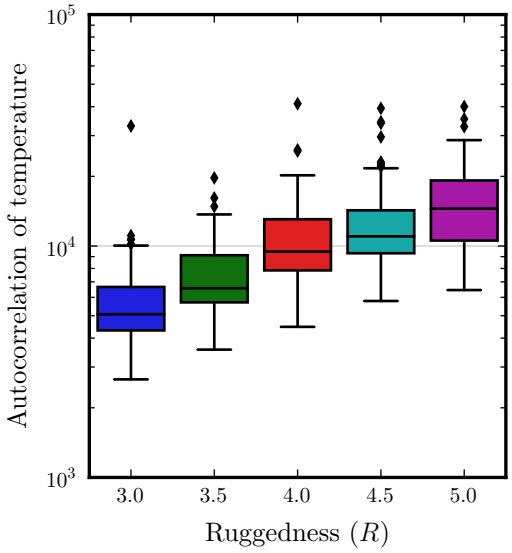


Fig. 2. (Color online) Box plots showing ruggedness versus classical hardness. We hold ρ and α fixed at 3 and 0.65, respectively, and vary the ruggedness R of the native Chimera problem. At each value of R we generated 100 instances; points indicate outliers. Classical hardness is measured using the autocorrelation of temperature. There is a clear positive correlation between ruggedness and classical hardness.

Hamiltonian. In Sect. 5, we attempt to deconvolve the impacts of ρ and R.

The ability to tune the ruggedness of the inputs by varying R, either by specifying $R = \rho$ and varying ρ, or by varying R independently, gives FCL problems an additional degree of utility, particularly when assessing the value of quantum tunneling and the potential of quantum annealing. Varying ρ and specifying $R = \rho$ makes problem generation simpler by reducing the number of free parameters whereas fixing ρ and varying R allows us to isolate the impact of ruggedness without altering the complexity of the logical problem.

3.4 Confirming correlation between ruggedness and classical hardness

We expect to see a positive correlation between ruggedness and classical hardness. Here we characterize classical hardness using the decorrelation time of an MCMC procedure. To validate this assumption we measure the decorrelation time for a parallel tempering (PT) procedure that uses the Metropolis algorithm in combination with the standard replica exchange rule;[46] we use the autocorrelation of temperature as our measure of decorrelation.[47] For more details of this method, see Appendix A.

Figure 2 illustrates the relationship between ruggedness and classical hardness. Confirming our intuition, FCL problems with greater ruggedness are characterized by greater classical hardness.

4. Software Solvers

The four software solvers we consider are GPU implementations of SA, QMC, and SVMC, and Selby's CPU implementation of HFS.[5,6]

Recent studies of D-Wave QPUs have not included GPU-based software solvers despite the fact that SA is very amenable to GPU implementation.[48] The addition of GPU solvers is a significant raising of the bar in terms of software competition, and means that solvers that can be implemented on GPUs have taken a leap forward relative to solvers that cannot. Run on modern hardware, our GPU-based algorithm implementations are roughly 1000 times faster than the corresponding single-core CPU implementations.

Mandrà et al.[4] analyzed the performance of a diverse set of solvers on the inputs of Denchev et al.[3] However the lack of GPU implementations of these solvers means that most are unlikely to be competitive in an absolute sense. Indeed, of the three classes of solvers that they study, only sequential algorithms, which they find to have the worst performance, have the massive parallelizability and low memory requirements that make efficient GPU implementations possible. It is also possible to implement SA in a field-programmable gate array (FPGA), but the additional speedup over GPU implementation is limited and generally not worth the increased cost of hardware.

In Appendix B we give further details of the software solvers and parameterizations we used. We also discuss algorithms we omitted because of prohibitive runtimes.

5. Optimization

We measure the expected time to solution (TTS) of different solvers on the inputs, calculated as

$$\mathrm{TTS} = \frac{\text{time per anneal}}{\text{ground state probability}}.$$

We consider only annealing times and exclude programming and readout times from our analysis as these are not part of the algorithms proper.

For a given value of ρ, we choose α to maximize the difficulty of the logical problem (see Fig. 1). For each selection of ρ and R, we generate 100 FCL problems at each problem size and solve each problem with each solver.

5.1 Varying ruggedness via logical complexity

In our first experiment, we vary ρ and set $R = \rho$. In this case the only free parameter ρ controls both the ruggedness and the logical complexity of the inputs. Time-to-solution plots are shown in Fig. 3. At the largest problem size, the D-Wave QPU is three orders of magnitude faster than the fastest software solver for each value of ρ. D-Wave's speedup over software peaks at 2600 times for $\rho = 4$.

©2019 The Physical Society of Japan

J. Phys. Soc. Jpn.
Downloaded from journals.jps.jp by Cornell University on 08/19/20

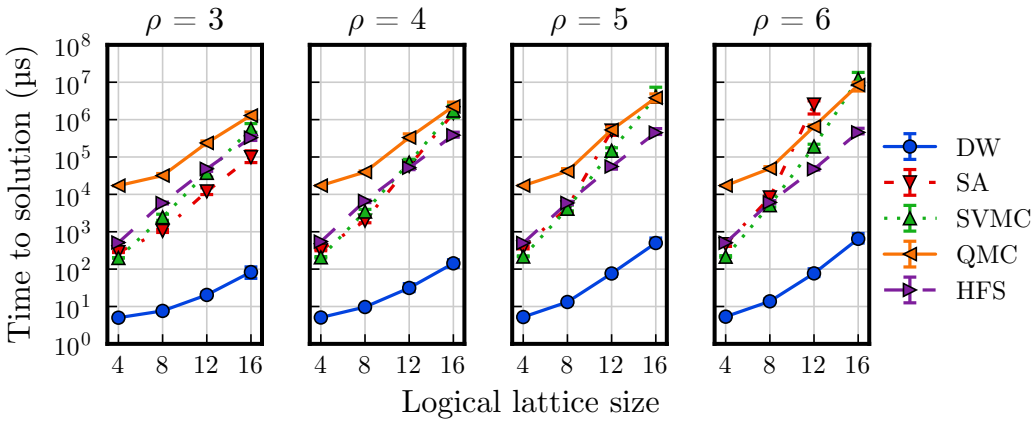


Fig. 3. (Color online) Time to solution for D-Wave and software solvers with range values $\rho \in \{3, 4, 5, 6\}$. For each value of ρ, α is chosen to maximize logical hardness. Shown are median values (over 100 inputs at each size) with 95% confidence intervals.

As ρ increases, the impact of local ruggedness increases as the logical Hamiltonian is compressed relative to the local wells induced by the clusters. The performance of SA drops off sharply while the performance of DW and QMC declines gracefully. The performance of HFS decreases only very slightly. HFS is not affected by the local ruggedness because it is tailored to the Chimera topology and uses updates that contain entire clusters; the performance degradation is due to the slight increase in logical problem hardness.

All solvers except HFS have strictly convex scaling curves because the anneal lengths are optimized for the largest problem size and are too long for the smaller problems. HFS does not use fixed-length anneals and ends up using shorter anneals on smaller inputs.

Though true scaling is masked by the inability to optimize parameters for smaller inputs,[1,49] we note that the performance of the D-Wave QPU scales at least as well as the software solvers between the two largest problem sizes.

5.2 *Varying ruggedness by scaling*

In our second experiment, we fix $\rho = 3$ and vary the ruggedness R. This keeps the logical complexity constant, allowing us to isolate the impact of ruggedness on the various solvers. Here we consider only the largest problem size having a 16×16 logical lattice.

Consistent with our findings when varying ρ, tuning the ruggedness directly by varying R increases difficulty dramatically for simulated annealing and less so for other solvers (see Fig. 4). Excluding HFS, whose behavior is constant in this example, the work required by a solver essentially scales according to its quantumness. The D-Wave QPU is most capable of dealing with ruggedness. QMC — the most faithful classical simulation of quantum annealing — comes next, followed by SVMC, which is a mean-field approximation to QMC. Bringing up the rear is SA, a simulation of a fully classical process.

The improved scaling (versus ruggedness) of QMC over SVMC indicates that crucial information is being lost in the mean-field approximation. The improved scaling of QA (i.e., the D-Wave QPU) over QMC may indicate that QMC is failing to faithfully simulate the dynamics of the QA processor, or it may simply be an artifact of our inability to use faster D-Wave anneals. This bears further investigation using future D-Wave QPUs with faster annealing times, again utilizing the tunable ruggedness of FCL problems.

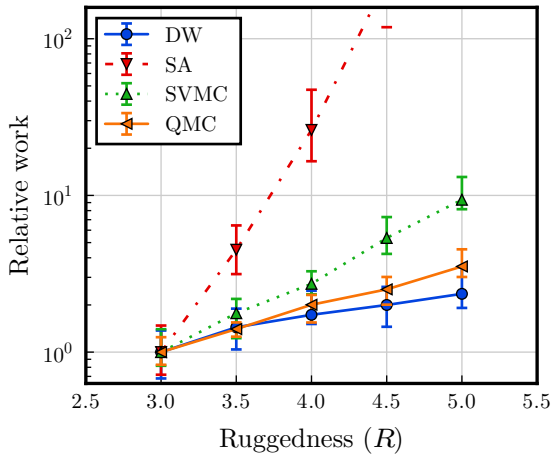


Fig. 4. (Color online) Ruggedness (increasing from left to right) versus relative work for various solvers. Relative work for each solver is calculated as TTS divided by median TTS at $R = 3.0$. The solvers have notably different responses to increasing ruggedness, with SA struggling the most, followed by SVMC, then QMC, then the D-Wave QPU. HFS deals with these energy barriers using exponential brute force; therefore the parameter R does not affect its performance. Markers indicate medians (over 100 inputs) and error bars indicate 95% confidence intervals for the median.

6. Sampling

The ability of an Ising solver to sample diverse optima has both practical and theoretical importance. Ground state sampling in combinatorial problems is the basis for construction of space-efficient SAT-based membership filters.[50,51] The associated complexity class, #P — the counting analog of NP — has been the subject of extensive research in theoretical computer science since the 1970s.[52] Sampling from the Boltzmann distribution, in which states with equal energy are sampled with equal probability, is of particular interest in machine learning. Boltzmann samples are used to train Boltzmann machines, a task known to be both hard and useful.[53]

While machine learning applications typically depend on finite-temperature Boltzmann sampling, using near-optimal states as well as optimal states, we focus on zero temperature

©2019 The Physical Society of Japan

J. Phys. Soc. Jpn.
Downloaded from journals.jps.jp by Cornell University on 08/19/20

sampling to simplify our investigation. This saves us from having an additional input parameter β — the inverse temperature — that we would have to either set arbitrarily or determine empirically. Empirical estimation of β can be challenging[54] and basing the target β on the output of a solver would arguably give that solver an unfair advantage.

6.1 *Sampling from all valleys*

The expected time required for a solver to find all ground states of a problem is known, both in the equiprobable case and the biased case.[55] In the case of an Ising spin problem, ground states often lie in connected valleys in Hamming space, and given one ground state in the cluster it is easy to find the rest. We therefore adopt a more practical metric based on the time required to sample all *valleys* of ground states.

In ground states of FCL problems, all clusters have their spins in agreement; therefore the distance between any two ground states in the native Hamming space is a multiple of 8. However, ground states can be adjacent (i.e., differ by a single spin) in the logical space. We define a *valley* as a set of ground states that are connected in the logical Hamming space. While it is nontrivial to move from one state to another in the same valley because of the single tall, thin energy barrier, it can still be done with a modest amount of postprocessing. We also note that, since FCL problems do not have fields, ground states come in antipodal pairs, and by extension so do valleys. For Hamiltonians with no fields, flipping all spins of a state does not change the energy. Therefore the antipode (negation) of any ground state is also a ground state. We treat each pair of antipodal valleys as a single valley since it is trivial to move from one to the other.

With valleys defined in this way, we define the time-to-all-valleys (TTAV) metric as the expected amount of annealing time required to draw at least one sample from each valley. This metric captures the hardest part of sampling from these distributions — finding ground states in every mode — and ensures a diverse set of solutions. With at least one sample from each valley, it is possible to find all ground states using only a modest amount of postprocessing. The TTAV metric is most meaningfully interpreted relative to TTS since hitting valleys directly depends on hitting ground states.

6.2 *Mining for interesting valley structure*

Sampling from all valleys is not always much harder than finding a single ground state — an input may have only a single valley or may have valleys that are all very close in Hamming space. We wish to generate inputs with multiple valleys that are well-separated. Sampling from distributions with multiple, well-separated valleys is particularly hard[56] and has important applications such as classification using deep Boltzmann machines.[57]

Because we define valleys as clusters of ground states in the *logical* space, analyzing the valley structure of an input is tractable. The largest logical graphs are 16×16 lattices having treewidth 16, so solving a logical problem using dynamic programming and returning some fixed number of ground states typically takes less than a second. This allows us to mine for inputs having interesting valley structure.

We quantify interesting valley structure using the distribution of spin overlap $P(q)$[36,37] at zero temperature, i.e., for two ground states sampled uniformly with replacement, what fraction of spins do they have in common? The random variable $P(q)$ takes values in the range $[-1, 1]$. For inputs without fields the distribution is symmetric about zero; we can therefore consider the distribution of the absolute value $P(|q|)$. We define the *mean overlap* as the expectation of $P(|q|)$. Inputs with mean overlap near 1 tend to resemble ferromagnets — if there are multiple valleys they will be close together. Inputs with lower mean overlap tend to have valleys that are well-separated.

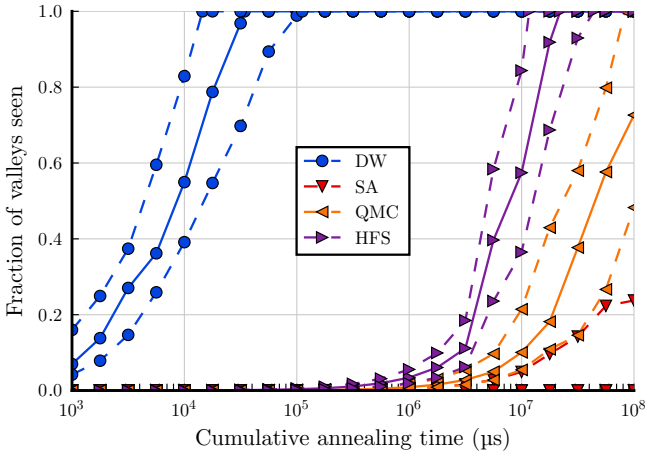


Fig. 5. (Color online) TTAV for various solvers. The x-axis shows elapsed annealing time and the y-axis shows the fraction of valleys that a solver has hit up to that point in time. Solid lines show medians (over 74 inputs) and dashed lines show the 25th and 75th percentiles.

Inputs that are hard to sample from have multiple valleys that are well-separated. We mine for such inputs as follows. First we reject any input with more than 1000 ground states, as these slow down our analysis and may be too easy. Second, we reject any input that does not have at least 4 valleys since we want valley collection to be nontrivial. Finally, we reject any input with a mean overlap of 0.7 or higher since we want valleys to be well-separated.

6.3 *Sampling results*

We generated problems at the 16×16 lattice size with $\alpha = 0.85$ and $\rho = R = 6$ and mined them for interesting valley structure as described above. We generated 50,000 inputs and rejected all but 74. This gave us an acceptance rate of roughly 0.15% of inputs. We sought to answer the question, after x seconds of annealing, what fraction of the valleys has each solver seen? For each problem we drew a number of samples according to the solver as follows:

D-Wave QPU:	100,000 samples at 5 μs (in batches of 100 per spin-reversal transform)
SA:	100,000 samples
QMC:	5000 samples
HFS:	5000 samples

6.3.1 *Time to all valleys*

Results for the TTAV metric are shown in Fig. 5. The 2000-qubit D-Wave QPU is the fastest of the solvers, hitting all valleys in a median time of roughly 30 ms. The fastest software competition was HFS, which hit all valleys in a median time of roughly 30 s.

©2019 The Physical Society of Japan

J. Phys. Soc. Jpn.
Downloaded from journals.jps.jp by Cornell University on 08/19/20

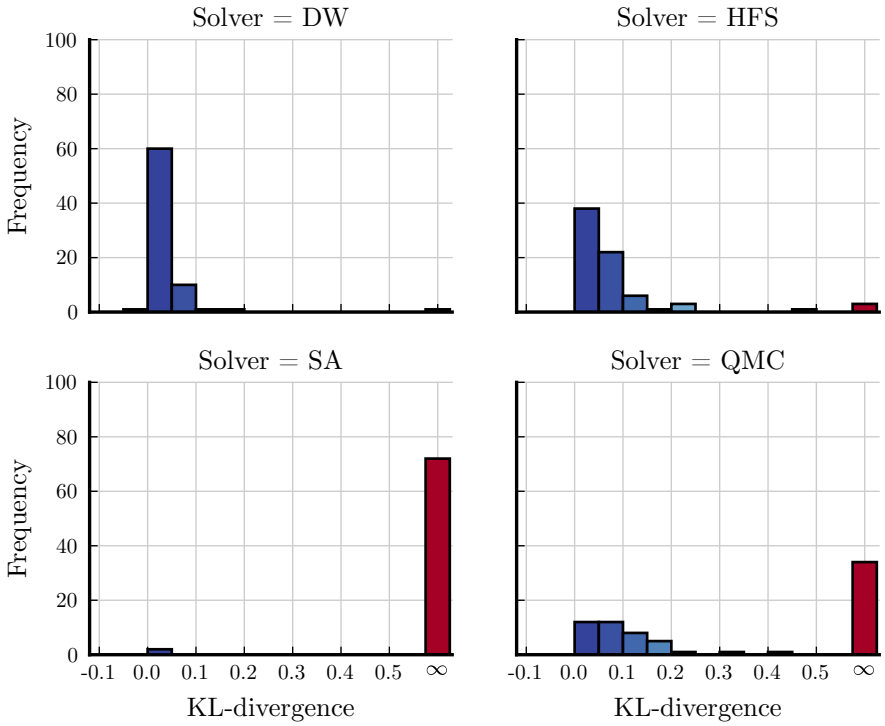


Fig. 6. (Color online) KL-divergence histograms. Shown are the empirical distributions (out of 74 inputs) of the KL-divergence achieved by each solver in estimating the valley distributions. Where the value is infinite, the solver failed to see one or more of the valleys. The D-Wave QPU had the best performance in this metric — even with three orders of magnitude less annealing time — followed by HFS, then QMC, then SA.

In certain situations quantum annealing in the transverse-field Ising model is subject to inherent sampling bias,[58–61] although that does not prove to be a significant problem here. While the slope of the D-Wave curve is slightly less steep than the HFS curve, indicating that its samples might be less diverse, the D-Wave QPU still manages to outperform the competition by about three orders of magnitude.

6.3.2 *KL-divergence of valley distributions*

The TTAV results shown in Fig. 5 fail to address a specific fear — that in a significant minority of inputs there are valleys that the D-Wave QPU would be simply unable to find due to quantum sampling bias. To address this, we calculate for each (solver, problem) pair the KL-divergence between empirical valley distributions and exact valley distributions (i.e., relative valley sizes). KL-divergence is an *asymmetric* measure of the distance between two probability distributions; we calculate it such that it is infinite if the solver fails to see all valleys, i.e.,

$$\mathrm{KLD} = \sum_{\text{valleys } v} P(v) \log \frac{P(v)}{\widehat{P}(v)},$$

where $P(v)$ is the true Boltzmann probability of valley v and $\widehat{P}(v)$ is the sample estimate of $P(v)$, conditioned on samples being ground states. This KL-divergence measure includes two types of error. First, there is a distributional error, since each solver samples from a distribution that differs from the Boltzmann distribution. Second, there is a sample size error, since our sample estimate has finite size and therefore differs from the solver's true distribution. In this context it is appropriate to include both types of error.

Figure 6 shows histograms of KL-divergence for the different solvers. For these FCL problems, fears of valleys suppressed by quantum sampling bias are unfounded. The D-Wave QPU has a superior KL-divergence distribution than any of the software solvers even when annealing for three orders of magnitude less time. On the single input for which the D-Wave KLD was infinite because at least one valley was never seen, it was also infinite for all other solvers.

6.3.3 *Raw error on model marginals*

The TTAV metric and valley distributions can be thought of as representing what sample quality would look like with postprocessing. We would also like a more raw metric that does not have this implicit postprocessing. For this we consider marginals of the zero-temperature Boltzmann distribution. Specifically, we consider the spin–spin expectations, i.e., for each coupler, what is the expected product of the two incident spins in the zero-temperature Boltzmann distribution? The L_1 error on these marginals (i.e., the empirical estimates of spin–spin expectations minus true expectations) is a well-established metric of interest in the study of undirected graphical models (see, e.g., Ref. 62).

Figure 7 shows the decay in error as more samples are taken. We measure errors in the logical space, so couplers within a cluster are ignored. Again, the D-Wave QPU is roughly three orders of magnitude faster than the fastest software solver. We expect that the error on marginals will decay to a certain point, then plateau, with the level of the plateau corresponding to the bias of the solver. As with the TTAV data, potential concerns about the bias of quantum annealers do not seem to play out here. The L_1 error of the D-Wave QPU is still decaying after 100,000 samples; at this point in time (0.5 s) the D-Wave QPU has achieved a median error of less than 1%, a value that software solvers fail to reach in 100 s to match.

©2019 The Physical Society of Japan

J. Phys. Soc. Jpn.
Downloaded from journals.jps.jp by Cornell University on 08/19/20

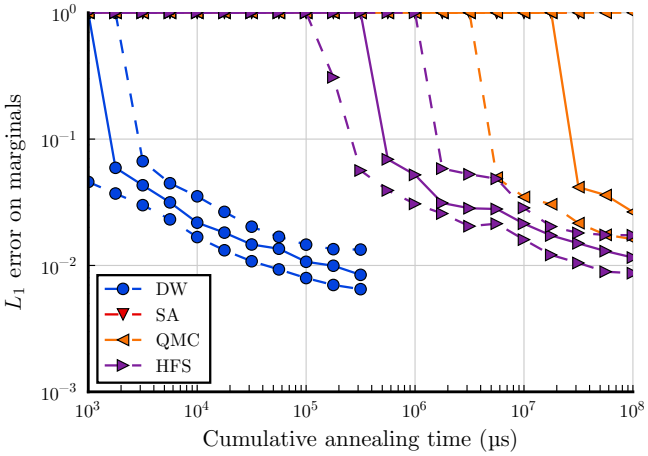


Fig. 7. (Color online) Elapsed annealing time versus L_1 error of marginal estimation for various solvers. Solid lines show medians (over 74 inputs) and dashed lines show the 25th and 75th percentiles. The D-Wave QPU achieves the same error as software solvers in roughly three orders of magnitude less time.

7. Conclusions

We have introduced a class of synthetic inputs on which to evaluate the performance of annealing-based QPUs, specifically the QPUs developed by D-Wave. This problem class is more representative of real-world problems and provides an alternative to the Google problems of Denchev et al.,[3] which were more highly tuned to highlight the utility of quantum tunneling.

The D-Wave QPU is up to 2600 times faster than all software solvers considered and typically on the order of 1000 times faster at the largest problem size. These software solvers now include GPU implementations of SA, QMC, and SVMC as well as a CPU implementation of HFS, making the competition much stronger than that analyzed by Denchev et al.[3] The set of software solvers we used was representative, but not exhaustive — in particular, we hope to include more of the solvers used by Mandrà et al.[4] in future studies.

These inputs have tunable ruggedness controlled either by the range parameter ρ or by the scaling parameter R. Of particular interest is the fact that QMC performed better relative to SA when the ruggedness is increased, and physical quantum annealing performed better still.

We also evaluated the 2000-qubit D-Wave QPU on the task of zero-temperature Boltzmann sampling, i.e., sampling uniformly from ground states. While concerns have been raised that quantum and analog sampling bias might make it difficult for quantum annealers to sample from Boltzmann distributions,[58–60] there was little evidence for such a struggle in this study. In several metrics considered, the 2000-qubit D-Wave QPU maintains its speed advantage of roughly three orders of magnitude and provides sample diversity that is as good as, or better than, the software competition.

Appendix A: Calculation of Decorrelation

The parallel tempering implementation we use to measure decorrelation time is parameterized by a sequence of n increasing inverse temperatures $\beta_i \in [0, \beta = 3]$. A replica sample is initialized randomly for each temperature. The replicas are then iterated, undertaking a random walk in temperature space combined with MCMC sweeps controlled by the energy landscape. At $\beta = 0$, the hottest replica, we replace the sample with a new random uniformly drawn sample on each iteration so that decorrelation is perfect. Inference on the lower temperature distributions is hardest. The quality of inference is limited by the time scale associated with the temperature random walk. For a sample to decorrelate at low temperature on a practical time scale, the random walk must pass through a high temperature state (where decorrelation is fast) and return to the low temperature state. The time scale is approximated by the integrated autocorrelation of the temperature index.[47]

In our setup, temperatures are selected independently for every instance, with β spanning $[0, 30]$ such that the replica exchange rate is equalized at close to 40% (no lower than 33%, no higher than 50%). Equalization of exchange rates is an intuitive and well-studied heuristic for temperature selection and is optimal in special cases.[63] We note that, as would be true in studying any problem class, better choices for temperatures and transition operators can lower the autocorrelation times relative to those presented. Equalization of exchange rates is achieved heuristically by iterating PT, refining the temperature set by linear interpolation of the log empirical exchange rates. To measure autocorrelation time, we undertake a long run of 600,000 sweeps, discarding a conservative portion (10%) of the initial samples which we took as sufficient for burn in (this assumption was tested self-consistently). We then extract the integrated autocorrelation time from the empirical values by an initial sequence estimator,[64] we average over the autocorrelations on the n available chains to reduce noise.

Table B·I. Specifications for classical processors used for software solvers.

Resource	CPU	GPU
Model	Intel®Xeon®CPU E5-2643 v3	NVIDIA®GeForce® GTX 1080
Clock rate	3.4 GHz	1.6 GHz
Cores	6	2560
Concurrent workers	6	1
Power	135 W	180 W

Appendix B: Details of Software Solvers

B.1 Classical hardware

Table B·I contains the specifications for the classical processors used. CPU algorithms were run single-threaded on one core each; multiple workers used cores concurrently for independent jobs. GPU algorithms are highly parallelized and each GPU job uses the entire GPU.

B.2 Included software solvers

Our experiments and analyses focus on four algorithms commonly used in comparisons with D-Wave QPUs — three that are highly amenable to GPU implementation and one that is highly tailored to the Chimera topology.

B.2.1 Simulated annealing

Simulated annealing[65] is a simulation of thermal annealing that is widely used as an optimization algorithm. It is the classical analog to quantum annealing. Since simulated annealing is a simple algorithm with very low memory

©2019 The Physical Society of Japan

J. Phys. Soc. Jpn.
Downloaded from journals.jps.jp by Cornell University on 08/19/20

requirements and a high degree of parallelizability, it is ideal for implementation on a GPU.

B.2.2 Quantum Monte Carlo

Quantum Monte Carlo, also known as simulated quantum annealing, is a classical approximation to quantum annealing. For the algorithm to work efficiently on a GPU, we implement the discrete time variant of QMC and fix the number of Trotter slices at 64 so that a worldline can be packed as bits in a word.

While the continuous time variant of QMC is a more faithful simulation of quantum annealing, in particular serving as a bias-free sampler that approaches the quantum Boltzmann distribution in the limit, it has been shown that discrete time QMC can have superior performance as an optimizer.[66)]

B.2.3 Spin vector Monte Carlo

Spin vector Monte Carlo (SVMC), also known as the O(2)-rotor model, is a mean-field approximation to QMC. SVMC can be thought of as falling between SA and QMC. Proposed for use as an approximation to D-Wave QPUs by Shin, Smith, Smolin, and Vazirani,[67)] it is also known as the SSSV algorithm. We use a GPU implementation of SVMC that is a minor modification of our implementation of SA.

B.2.4 Hamze–de Freitas–Selby

The HFS algorithm optimizes by repeatedly optimizing the spins in a low-treewidth induced subgraph of the input, subject to the rest of the input being fixed. The subgraph over which the input is optimized changes at each step. The HFS algorithm is a greedy search algorithm in which reassignment of many variables is considered at once. We used Selby's implementation[5,6)] that is heavily tailored to the Chimera topology; we modified this solver to return each stopping state for consistency with the other heuristic solvers. We note that the HFS algorithm cannot be efficiently implemented on GPU because the memory requirements are too high.

B.3 Excluded software solvers

In addition to these four algorithms, we considered several other software solvers that were prohibitively slow; due to limited time and resources it was not feasible to perform the long software runs needed to optimize parameters and determine ideal performance.

B.3.1 Nontailored HFS

We tested an implementation of HFS that, rather than using subgraphs tailored to the Chimera topology, is topology-agnostic and generates subgraphs dynamically. This nontailored version of HFS performed far worse than Selby's tailored implementation, to the point where we failed to hit ground states in the largest problems. The failure of this algorithm highlights the extent to which Selby's HFS implementation, and specifically the hardcoded subgraphs to update, exploit the sparsity and modularity of the Chimera topology.[4)] It is very likely that this type of exploitation will be impossible in future quantum annealer topologies.[3)]

B.3.2 Wolff cluster Monte Carlo

The Wolff algorithm[68)] dynamically detects clusters of spins that should be flipped together. We used a modified implementation that considers the potential change in energy when deciding whether to flip a cluster, similar to Venturelli et al.[69)] This algorithm would, at first glance, be ideal for FCL problems due to the crucial role of clusters. However, finding clusters is slow and our CPU implementation was not competitive with other solvers. Note that the Wolff algorithm is not particularly amenable to GPU implementation; such implementations exist for topologies such as lattices[70)] but they achieve only modest speedups over CPU implementations.

B.3.3 Parallel tempering

Parallel tempering runs multiple replicas of a Monte Carlo simulation at different temperatures in parallel, and can exchange information between replicas according to certain exchange rules. It is the algorithm of choice for approximately calculating features and statistics of an energy landscape when exact calculation is prohibitive. Our GPU implementation used 64 replicas, with replicas of a spin packed bitwise into a word, similar to our implementation of QMC. Parallel tempering is more powerful than simulated annealing, but in this case proved to be uncompetitive due to the increased cost of each step.

B.3.4 PT-ICM

In vanilla parallel tempering, replica exchange steps are combined with single-spin Monte Carlo updates. However, replica exchanges can be combined with other types of updates. Isoenergetic cluster move (ICM) updates can be combined with replica exchange and simple Monte Carlo updates to obtain PT-ICM. This technique was introduced by Houdayer[12)] and is a variant of the cluster replica Monte Carlo method introduced by Swendsen and Wang.[11)] We implemented PT-ICM and found that its scaling was similar to Selby's implementation of HFS, but absolute performance was an order of magnitude slower even when using a precomputed ideal temperature ladder (i.e., set of β values) specific to each input.

The advantage that PT-ICM has over Selby's implementation of HFS is that PT-ICM detects clusters dynamically and is not tailored to the underlying topology. Because of this, there may be a misconception that PT-ICM will be future proof against denser quantum processor topologies. However, problems on denser topologies will have smaller site-percolation thresholds,[71)] and the effectiveness of PT-ICM depends crucially on a large site-percolation threshold.[72)] Thus increased processor density will erode the computational value of cluster updates and consequently the advantage of PT-ICM over vanilla PT.

B.4 Parameter tuning

For the D-Wave QPU, optimal performance at all problem sizes was achieved at the minimum allowed annealing time of 5 μs. It therefore makes sense to optimize the parameters of software solvers only at the largest problem size; optimizing on a per-size basis would only make scaling look worse for the software solvers. Since we cannot properly optimize the performance of all solvers at all problem sizes, we focus on results at the largest problem size and take scaling results with a grain of salt.

©2019 The Physical Society of Japan

J. Phys. Soc. Jpn.
Downloaded from journals.jps.jp by Cornell University on 08/19/20

For SA, we chose values of β that increase linearly from 0.01 to 3 and used 10^5 sweeps. For QMC, we used a fixed β of 30 and 10^4 sweeps with the transverse field $A(t)$ decreasing linearly from 1 to 0 and the longitudinal field $B(t)$ increasing linearly from 0 to 1. For SVMC we again used a β of 30 and the same annealing schedule, but used 10^5 sweeps. These values of β were chosen to optimize performance. For HFS, we used Selby's strategy GS-TW2.[5)] For the D-Wave QPU, we use the minimum annealing time of 5 μs.

*jking@dwavesys.com

1) T. Rønnow, Z. Wang, J. Job, S. Boixo, S. Isakov, D. Wecker, J. Martinis, D. Lidar, and M. Troyer, Science **345**, 420 (2014).
2) H. Katzgraber, F. Hamze, and R. Andrist, Phys. Rev. X **4**, 021008 (2014).
3) V. S. Denchev, S. Boixo, S. V. Isakov, N. Ding, R. Babbush, V. Smelyanskiy, J. Martinis, and H. Neven, Phys. Rev. X **6**, 031015 (2016).
4) S. Mandrà, Z. Zhu, W. Wang, A. Perdomo-Ortiz, and H. G. Katzgraber, Phys. Rev. A **94**, 022337 (2016).
5) A. Selby, arXiv:1409.3934.
6) A. Selby, QUBO-Chimera. https://github.com/alex1770/QUBO-Chimera, 2013. GitHub repository.
7) F. Hamze and N. de Freitas, Proceedings of the 20th Conference on Uncertainty in Artificial Intelligence, 2004, p. 243.
8) L. Bieche, J. P. Uhry, R. Maynard, and R. Rammal, J. Phys. A **13**, 2553 (1980).
9) S. E. Schaeffer, Comput. Sci. Rev. **1**, 27 (2007).
10) S. Fortunato, Phys. Rep. **486**, 75 (2010).
11) R. H. Swendsen and J.-S. Wang, Phys. Rev. Lett. **57**, 2607 (1986).
12) J. J. Houdayer, Eur. Phys. J. B **22**, 479 (2001).
13) Z. Zhu, A. J. Ochoa, and H. G. Katzgraber, Phys. Rev. Lett. **115**, 077201 (2015).
14) J. King, S. Yarkoni, J. Raymond, I. Ozfidan, A. D. King, M. M. Nevisi, J. P. Hilton, and C. C. McGeoch, arXiv:1701.04579.
15) S. Mandrà and H. G. Katzgraber, Quantum Sci. Technol. **3**, 04LT01 (2018).
16) T. Albash and D. A. Lidar, Phys. Rev. X **8**, 031016 (2018).
17) P. I. Bunyk, E. M. Hoskinson, M. W. Johnson, E. Tolkacheva, F. Altomare, A. J. Berkley, R. Harris, J. P. Hilton, T. Lanting, A. J. Przybysz, and J. Whittaker, IEEE Trans. Appl. Supercond. **24**, 1 (2014).
18) N. G. Dickson, M. W. Johnson, M. H. Amin, R. Harris, F. Altomare, A. J. Berkley, P. Bunyk, J. Cai, E. M. Chapple, P. Chavez, F. Cioata, T. Cirip, P. deBuen, M. Drew-Brook, C. Enderud, S. Gildert, F. Hamze, J. P. Hilton, E. Hoskinson, K. Karimi, E. Ladizinsky, N. Ladizinsky, T. Lanting, T. Mahon, R. Neufeld, T. Oh, I. Perminov, C. Petroff, A. Przybysz, C. Rich, P. Spear, A. Tcaciuc, M. C. Thom, E. Tolkacheva, S. Uchaikin, J. Wang, A. B. Wilson, Z. Merali, and G. Rose, Nat. Commun. **4**, 1903 (2013).
19) R. Harris, M. W. Johnson, T. Lanting, A. J. Berkley, J. Johansson, P. Bunyk, E. Tolkacheva, E. Ladizinsky, N. Ladizinsky, T. Oh, F. Cioata, I. Perminov, P. Spear, C. Enderud, C. Rich, S. Uchaikin, M. C. Thom, E. M. Chapple, J. Wang, B. Wilson, M. H. S. Amin, N. Dickson, K. Karimi, B. Macready, C. J. S. Truncik, and G. Rose, Phys. Rev. B **82**, 024511 (2010).
20) M. W. Johnson, M. H. S. Amin, S. Gildert, T. Lanting, F. Hamze, N. Dickson, R. Harris, A. J. Berkley, J. Johansson, P. Bunyk, E. M. Chapple, C. Enderud, J. P. Hilton, K. Karimi, E. Ladizinsky, N. Ladizinsky, T. Oh, I. Perminov, C. Rich, M. C. Thom, E. Tolkacheva, C. J. S. Truncik, S. Uchaikin, J. Wang, B. Wilson, and G. Rose, Nature **473**, 194 (2011).
21) T. Lanting, A. J. Przybysz, A. Yu. Smirnov, F. M. Spedalieri, M. H. Amin, A. J. Berkley, R. Harris, F. Altomare, S. Boixo, P. Bunyk, N. Dickson, C. Enderud, J. P. Hilton, E. Hoskinson, M. W. Johnson, E. Ladizinsky, N. Ladizinsky, R. Neufeld, T. Oh, I. Perminov, C. Rich, M. C. Thom, E. Tolkacheva, S. Uchaikin, A. B. Wilson, and G. Rose, Phys. Rev. X **4**, 021041 (2014).
22) N. Robertson and P. D. Seymour, J. Algorithms **7**, 309 (1986).
23) S. Boixo, T. Albash, F. M. Spedalieri, N. Chancellor, and D. A. Lidar, Nat. Commun. **4**, 2067 (2013).
24) T. Kadowaki and H. Nishimori, Phys. Rev. E **58**, 5355 (1998).
25) E. Farhi, J. Goldstone, S. Gutmann, J. Lapan, A. Lundgren, and D. Preda, Science **292**, 472 (2001).
26) S. Boixo, V. N. Smelyanskiy, A. Shabani, S. V. Isakov, M. Dykman, V. S. Denchev, M. H. Amin, A. Y. Smirnov, M. Mohseni, and H. Neven, Nat. Commun. **7**, 10327 (2016).
27) J. King, S. Yarkoni, M. Mohammadi Nevisi, J. P. Hilton, and C. C. McGeoch, arXiv:1508.05087.
28) E. Weinberger, Biol. Cybern. **63**, 325 (1990).
29) V. K. Vassilev, T. C. Fogarty, and J. F. Miller, Evol. Comput. **8**, 31 (2000).
30) W. S. Kendall, F. Liang, and J.-S. Wang, *Markov Chain Monte Carlo: Innovations and Applications* (World Scientific, Singapore, 2005) Vol. 7.
31) W. Janke, *Rugged Free Energy Landscapes: Common Computational Approaches to Spin Glasses, Structural Glasses and Biological Macromolecules* (Springer, Berlin/Heidelberg, 2007) Vol. 736.
32) S. Kauffman and S. Levin, J. Theor. Biol. **128**, 11 (1987).
33) S. A. Kauffman and E. D. Weinberger, J. Theor. Biol. **141**, 211 (1989).
34) E. Weinberger, NP Completeness of Kauffman's N-k Model, A Tuneable Rugged Fitness Landscape (1996).
35) M. Pelikan, K. Sastry, D. E. Goldberg, M. V. Butz, and M. Hauschild, Proceedings of the 11th Annual Conference on Genetic and Evolutionary Computation, 2009, p. 851.
36) B. Yucesoy, J. Machta, and H. G. Katzgraber, Phys. Rev. E **87**, 012104 (2013).
37) H. G. Katzgraber, H. Firas, Z. Zhu, A. J. Ochoa, and H. Munoz-Bauza, Phys. Rev. X **5**, 031026 (2015).
38) Z. Zhu, A. Feldman, S. Isakov, H. G. Katzgraber, S. Mandrà, H. Munoz-Bauza, A. Ochoa, A. Ozaeta, and A. Perdomo-Ortiz, Adiabatic Quantum Computing Conference, 2016.
39) H. G. Katzgraber, S. Boixo, V. V. Denchev, F. Hamze, S. Isakov, S. Mandrà, O. Melchert, H. Neven, A. Perdomo-Ortiz, S. Schnabel, A. Barzegar, J. Chancellor, C. Fang, R. Lawrence, H. Munoz-Bauza, A. Ochoa, W. Wang, and Z. Zhu, Adiabatic Quantum Computing Conference, 2016.
40) C. P. Gomes and B. Selman, Artif. Intell. **126**, 43 (2001).
41) S. Venkatasubramanian and R. W. Vuduc et al., Proceedings of the 23rd International Conference on Supercomputing, 2009, p. 244.
42) D. Grewe and M. F. P. O'Boyle, International Conference on Compiler Construction, 2011, p. 286.
43) I. Hen, J. Job, T. Albash, T. F. Rønnow, M. Troyer, and D. A. Lidar, Phys. Rev. A **92**, 042325 (2015).
44) A. D. King, T. Lanting, and R. Harris, arXiv:1502.02098.
45) K. L. Pudenz, T. Albash, and D. A. Lidar, Phys. Rev. A **91**, 042302 (2015).
46) K. Hukushima and K. Nemoto, J. Phys. Soc. Jpn. **65**, 1604 (1996).
47) R. A. Baños, A. Cruz, L. A. Fernandez, J. M. Gil-Narvion, A. Gordillo-Guerrero, M. Guidetti, A. Maiorano, F. Mantovani, E. Marinari, V. Martin-Mayor, J. Monforte-Garcia, A. M. Sudupe, D. Navarro, G. Parisi, S. Perez-Gaviro, J. J. Ruiz-Lorenzo, S. F. Schifano, B. Seoane, A. Tarancon, R. Tripiccione, and D. Yllanes, J. Stat. Mech. **2010**, P06026 (2010).
48) A. M. Ferreiro, J. A. García, J. G. López-Salas, and C. Vázquez, J. Glob. Optim. **57**, 863 (2013).
49) M. H. Amin, Phys. Rev. A **92**, 052323 (2015).
50) S. A. Weaver, K. J. Ray, V. W. Marek, A. J. Mayer, and A. K. Walker, J. Satisfiability Boolean Model. Comput. **8**, 129 (2014).
51) A. Douglass, A. D. King, and J. Raymond, International Conference on Theory and Applications of Satisfiability Testing, 2015, p. 104.
52) L. G. Valiant, Theor. Comput. Sci. **8**, 189 (1979).
53) P. M. Long and R. Servedio, Proceedings of the 27th International Conference on Machine Learning (ICML-10), 2010, p. 703.
54) J. Raymond, S. Yarkoni, and E. Andriyash, Frontiers in ICT **3**, 23 (2016).
55) P. Flajolet, D. Gardy, and L. Thimonier, Discrete Appl. Math. **39**, 207 (1992).
56) R. M. Neal, Stat. Comput. **6**, 353 (1996).
57) N. Srivastava and R. R. Salakhutdinov, Advances in Neural Information Processing Systems, 2012, p. 2222.

©2019 The Physical Society of Japan

J. Phys. Soc. Jpn.
Downloaded from journals.jps.jp by Cornell University on 08/19/20

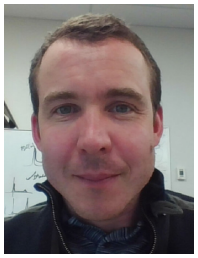
58) T. Albash, W. Vinci, A. Mishra, P. A. Warburton, and D. A. Lidar, Phys. Rev. A **91**, 042314 (2015).
59) Y. Matsuda, H. Nishimori, and H. G. Katzgraber, New J. Phys. **11**, 073021 (2009).
60) S. Mandrà, Z. Zhu, and H. G. Katzgraber, Phys. Rev. Lett. **118**, 070502 (2017).
61) A. D. King, E. Hoskinson, T. Lanting, E. Andriyash, and M. H. Amin, Phys. Rev. A **93**, 052320 (2016).
62) A. Globerson and T. Jaakkola, in *Proc. the Eleventh Int. Conf. on Artificial Intelligence and Statistics, Proc. Machine Learning Research*, eds. M. Meila and X. Shen, 2007, Vol. 2, p. 131.
63) A. Kone and D. A. Kofke, J. Chem. Phys. **122**, 206101 (2005).
64) W. Janke, Quantum Simulations of Complex Many-Body Systems: From Theory to Algorithms **10**, 423 (2002).
65) S. Kirkpatrick, C. Gelatt, Jr., and M. Vecchi, Science **220**, 671 (1983).
66) B. Heim, T. F. Rønnow, S. V. Isakov, and M. Troyer, Science **348**, 215 (2015).
67) S. W. Shin, G. Smith, J. A. Smolin, and U. Vazirani, arXiv:1401.7087.
68) U. Wolff, Phys. Rev. Lett. **62**, 361 (1989).
69) D. Venturelli, S. Mandrà, S. Knysh, B. O'Gorman, R. Biswas, and V. Smelyanskiy, Phys. Rev. X **5**, 031040 (2015).
70) Y. Komura and Y. Okabe, J. Comput. Phys. **231**, 1209 (2012).
71) B. Bollobas and O. Riordan, *Percolation* (Cambridge University Press, Cambridge, U.K., 2006).
72) Z. Zhu, C. Fang, and H. G. Katzgraber, arXiv:1605.09399.



James King received his Ph.D. from McGill University in 2010, specializing in the design and analysis of probabilistic algorithms. Subsequently he worked as a postdoctoral researcher at the University of Oxford investigating the structure of combinatorial energy landscapes and has worked as an algorithm researcher at D-Wave Systems since 2013. His current research focuses on the design of hybrid quantum/classical algorithms.



Sheir Yarkoni received a B.Sc. in Computer Science in Physics and an M.Sc. in Physics from McGill University in Montreal. He joined D-Wave in 2014 and currently works with customers to solve industrial optimization problems on D-Wave quantum computers. In 2017 he began his Ph.D. in Computer Science at Leiden University.



Jack Raymond received his Doctor of Applied Mathematics from Aston University. Following research positions at Hong Kong UST and La Sapienza University of Rome he joined D-Wave in 2014 and is currently a senior algorithms researcher. His research focuses on sampling applications of quantum annealing, statistical physics and variational inference.



Isil Ozfidan received her Ph.D. in Physics from the University of Ottawa. Following a Postdoctoral position at the University of Alberta, she joined D-Wave in 2016 and is currently a Senior Research Scientist. Her research focuses on quantum chemistry and qubit modeling.



Andrew D. King received his Ph.D. from McGill University and worked as a postdoctoral researcher at Columbia University and Simon Fraser University before joining D-Wave Systems in 2013. With a background in graph theory and algorithms, his research has recently focused on performance evaluation and enhancement in quantum annealing processors.



Mayssam Mohammadi Nevisi received his Ph.D. in Computer Science from Simon Fraser University in Burnaby, Canada, specializing in graph partitioning algorithms. Previously he received a B.Sc. in Computer Engineering and an M.Sc. in Computer Science from Sharif University in Tehran, Iran.



Jeremy P. Hilton joined D-Wave directly after obtaining his B.Sc. in Physics from the University of British Columbia and is currently D-Wave's Senior VP of Technology and Applications. Since 2004 he and his team of engineers and scientists have created a world-class R&D facility and built the world's first scalable quantum processors. Under Mr. Hilton's leadership, D-Wave has successfully delivered four generations of quantum computing products, executing an aggressive product roadmap and developing a strong base of customers through system installations and online access. Mr. Hilton is a named inventor on 36 granted US patents.



Catherine McGeoch received her Ph.D. in Computer Science from Carnegie Mellon University and for 29 years was on the faculty at Amherst College. Since 2013 she worked at D-Wave Systems. She co-founded the DIMACS Challenge series and the Workshop on Algorithm Engineering and Experiments (now called ALENEX). Her research interests are in experimental algorithmics, especially evaluation of heuristics for NP-hard problems, and in performance analysis of quantum annealing processors. She is past Editor in Chief of the ACM Journal on Experimental Algorithmics. She is the author of A Guide to Experimental Algorithmics and of Adiabatic Quantum Computation and Quantum Annealing: Theory and Practice.

©2019 The Physical Society of Japan